

Documentation for Serializable Types v1.0.0

Tested with Unity 2022.3+

Full offline documentation

Getting started

Wrapper types

This asset contains 4 wrapper classes: SerializableDictionary, SerializableSet, SerializableStack, SerializableQueue. Each wrapper class is generic and contains Container type for a respective class.

Each class implements editing and access functions for their respective Collection. All of the classes are serializable through Unity's Serializer and are compatible with JsonUtility.

Usage example

```
public class DictionaryShow : MonoBehaviour {
    [Header("Dictionary views")]
    [SerializeField] private SerializableDictionary<string, string> _stringTest;

    public void PrintStringDictionary() {
        foreach (var kv in _stringTest.Data) {
            Debug.Log($"{kv.Key} - {kv.Value}");
        }
    }
}
```

Subclass selector

In addition this asset also contains: SubclassSelector - an attribute for a polymorphic subclass selector for SerializeReference.

To use simply add this attribute to your variable like so:

```
[SerializeReference, SubclassSelector] private BaseClass _test;
```

Alternatively you can use RefWrapper to wrap your variable type and get similar result:

```
[SerializeField] private RefWrapper<BaseClass> _test;
```

RefWrapper can be use with Serializable wrapper types like this:

```
[SerializeField] private SerializableDictionary<string, RefWrapper<BaseClass>> _test;
```

More examples can be found in Examples folder.

API reference

SerializableDictionary

This class holds the Dictionary and a List of KeyValue pairs in it.

Fields

Data (IRReadOnlyDictionary) – for read operations.

Functions

bool Add(key, value) – Similar to Dictionary.Add(key, value). Updates serialized list.
Returns – false if key already exists true otherwise.

bool RemoveKey(key) – Similar to Dictionary.RemoveKey(key). Removes all of the occurrences in serialized list. Has time complexity of $O(n)$.
Returns – true if key removed, false if doesn't exist.

void Clear() – Clears both underlying Dictionary and serialized List.

SerializedSet

This class holds the HashSet and a List.

Fields

Data (ICollection) – for read operations.

Functions

bool Contains(value) – Proxy for HashSet.Contains(value).

Returns – true if HashSet contains value, false otherwise.

bool Add(value) – Similar to HashSet.Add(value). Updates serialized list.

Returns – false if value already exists true otherwise.

bool Remove(value) – Similar to HashSet.Remove(value). Removes all of the occurrences in serialized list. Has time complexity of $O(n)$.

Returns – true if value removed, false if doesn't exist.

void Clear() – Clears both underlying HashSet and serialized List.

SerializableStack

This class holds the Stack and a List.

Fields

Count (int) – number of elements in the Stack.

Functions

void Push(value) – Similar to Stack.Push(value). Updates serialized List.

bool TryPop(out result) – Similar to Stack.TryPop(out result). On success removes element from serialized List.

Returns – false if stack is empty, true otherwise.

bool TryPeek(out result) – Proxy for Stack.TryPeek(out result).

Returns – false if stack is empty, true otherwise.

T Pop() – Similar to Stack.Pop(). Removes the last element from serialized List. Will throw exception on empty Stack.

Returns – Removed element.

T Peek() – Proxy for Stack.Peek(). Will throw exception on empty Stack.

Returns – Next element in the Stack.

void Clear() – Clears both underlying Stack and serialized List.

SerializableQueue

This class holds the Queue and a List.

Fields

Count (int) – number of elements in the Queue.

Functions

void Enqueue(value) – Similar to Queue.Enqueue(value). Updates serialized List.

bool TryDequeue(out result) – Similar to Queue.TryDequeue(out result). On success removes element from serialized List.

Returns – false if queue is empty, true otherwise.

bool TryPeek(out result) – Proxy for Queue.TryPeek(out result).

Returns – false if queue is empty, true otherwise.

T Dequeue() – Similar to Queue.Dequeue(). Removes the first element from serialized List. Will throw exception on empty Queue.

Returns – Removed element.

T Peek() – Proxy for Queue.Peek(). Will throw exception on empty Queue.

Returns – Next element in the Queue.

void Clear() – Clears both underlying Queue and serialized List.

RefWrapper

Wrapper class for SerializeReference variables. Has implicit converters to and from wrapped type.

Fields

Value { set; get; } - accessor for an underlying type.

SubclassSelector

Can be used with [SerializeReference] to create a polymorphic subclass picker in the Inspector.

Example

```
[SerializeReference, SubclassSelector] BaseType _test;
```

Editor features

Effect

EffectDamage

▼ Data

Damage5

Incorrect use

Type not supported

List of references

Effect List2

Element 0

EffectDamage

▼ Data

Damage5

Element 1

EffectStun

▼ Data

Stun Time2

+ -

Reference wrapper class

Wrapper Test

ValueNull

SubclassSelector custom drawers.

Dictionary views

String Test4

Skibidi

KeySkibidi

ValueToilet

More

KeyMore

ValueBullets

More

KeyMore

ValueBullets

More

KeyMore

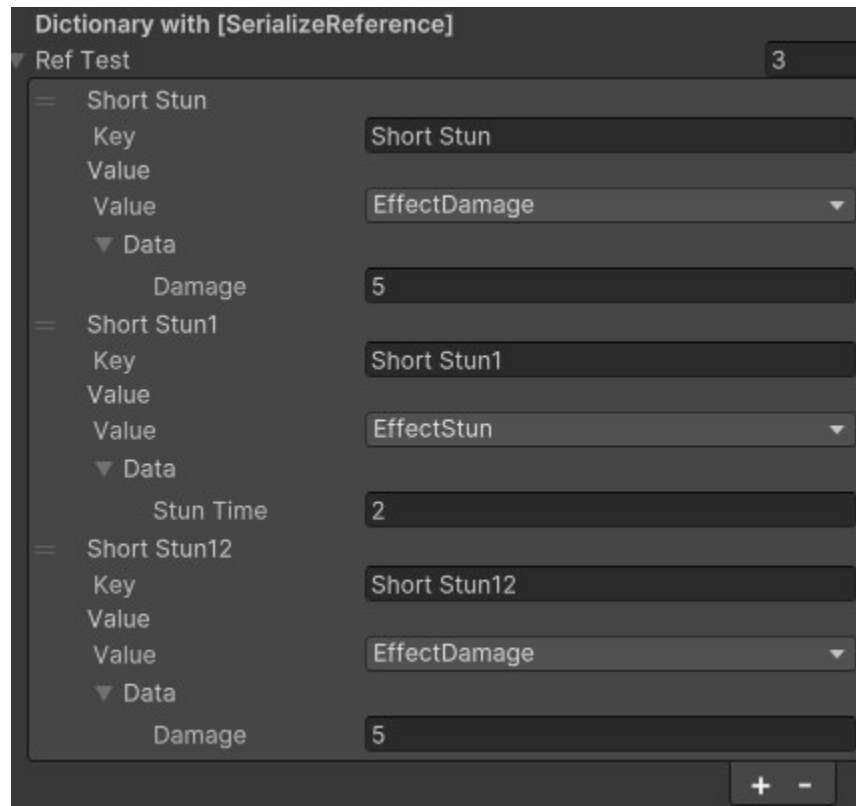
ValueBullets

+ -

⚠

This dictionary has duplicate keys

Custom Dictionary drawer with duplicate key warning.



SubclassSelector inside SerializableDictionary.

JsonUtility compatibility

All of the wrapper classes are serializable by Unity's serializer and JsonUtility. To be serialized by JsonUtility, the wrapper variable should be marked with [SerializeField] All of the wrapper classes can also be serialized on their own.

Examples

```
{  
    var dict = new SerializableDictionary<string, int>();  
  
    var json = JsonUtility.ToJson(dict);  
}
```

```
public class Inventory {
```



```
    [SerializeField] SerializableSet<int> _items;  
    [SerializeField] SerializableStack<int> _purse;  
}  
{  
    var inventory = new Inventory();  
  
    var json = JsonUtility.ToJson(inventory);  
}
```

Performance

This asset is not intended to be used in a performance heavy situations. Each wrapper class has duplicate data, requiring double the memory. And editing functions have $O(n)$ complexity. This asset is for editor viewing/editing and JSON serialization with direct compatibility with Unity.